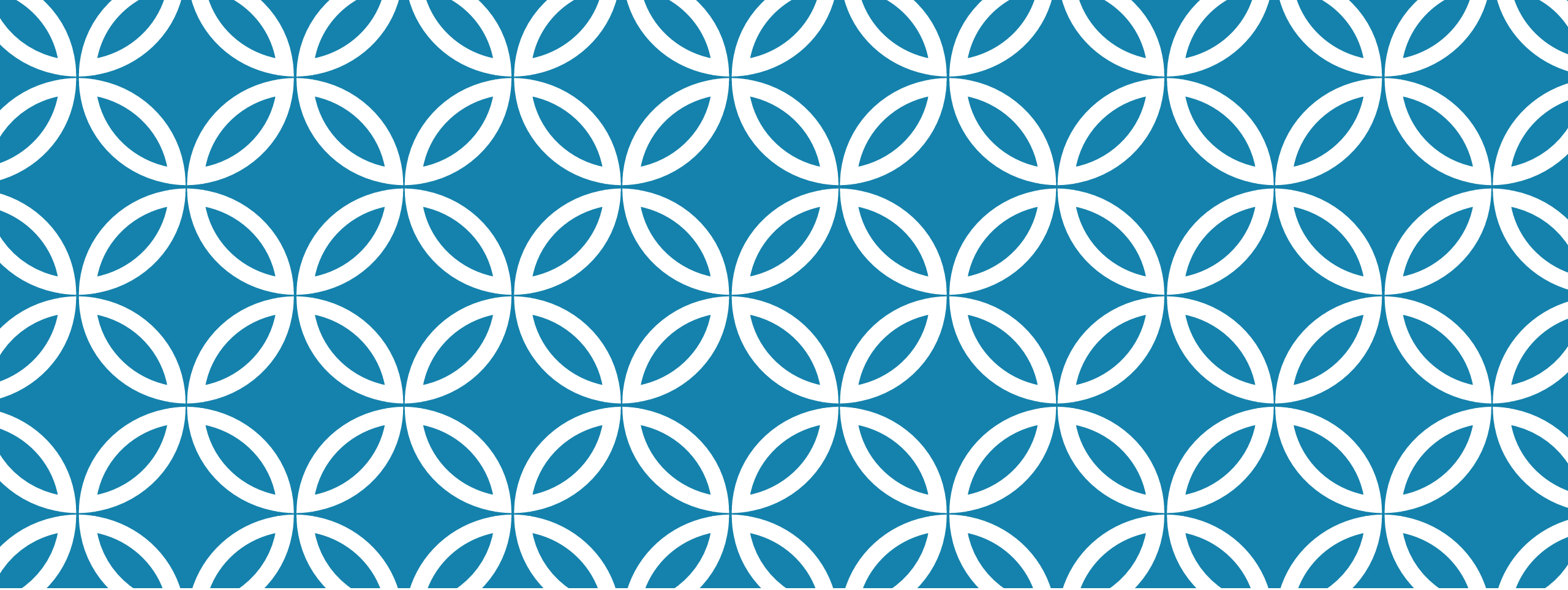


CSE455 - SOFTWARE TESTING

Chapter 5

DYNAMIC TESTING

Involves running the software on a computer

Requires an executable test object

Input data is fed to the test driver before execution

Often needs a *unit test framework* for component testing like JUnit (xUnit)

Uses black-box, white-box, and experience-based techniques

Main goal is to check if requirements are fulfilled and failures found

PURPOSE OF DT

Confirms that requirements are met

Identify any deviation among requirements and implementation

Use as little effort as possible to test as many requirements and identify as many failures as possible

STEPS IN DT

- Define the test conditions and preconditions, and objective you wish to achieve
- Specify individual test cases
- Define the test execution schedule

Degree of formality of the process depends on the maturity of development and test processes, time constraints and skills of the team members

Traceability between individual requirements and their corresponding test cases enables us to analyze the *impact of changes* to requirements on the testing process

- Designing new test cases, discarding redundant test cases, modifying existing test cases

STEPS IN DT CONT..

Expected results must be defined and documented before test is executed. Violation of this guideline results in an incorrect test result might be interpreted as fault-free, thus allowing a system failure to go unnoticed

Test Execution Schedule

- List test cases thematically according to their objectives
- Test priorities
- Technical/Logical dependencies between tests
- Assignment of test cases to individual testers

UNIT TEST FRAMEWORK

Required for testing non-standalone components

An environment for dynamic testing is created by emulating the context of the unit under test

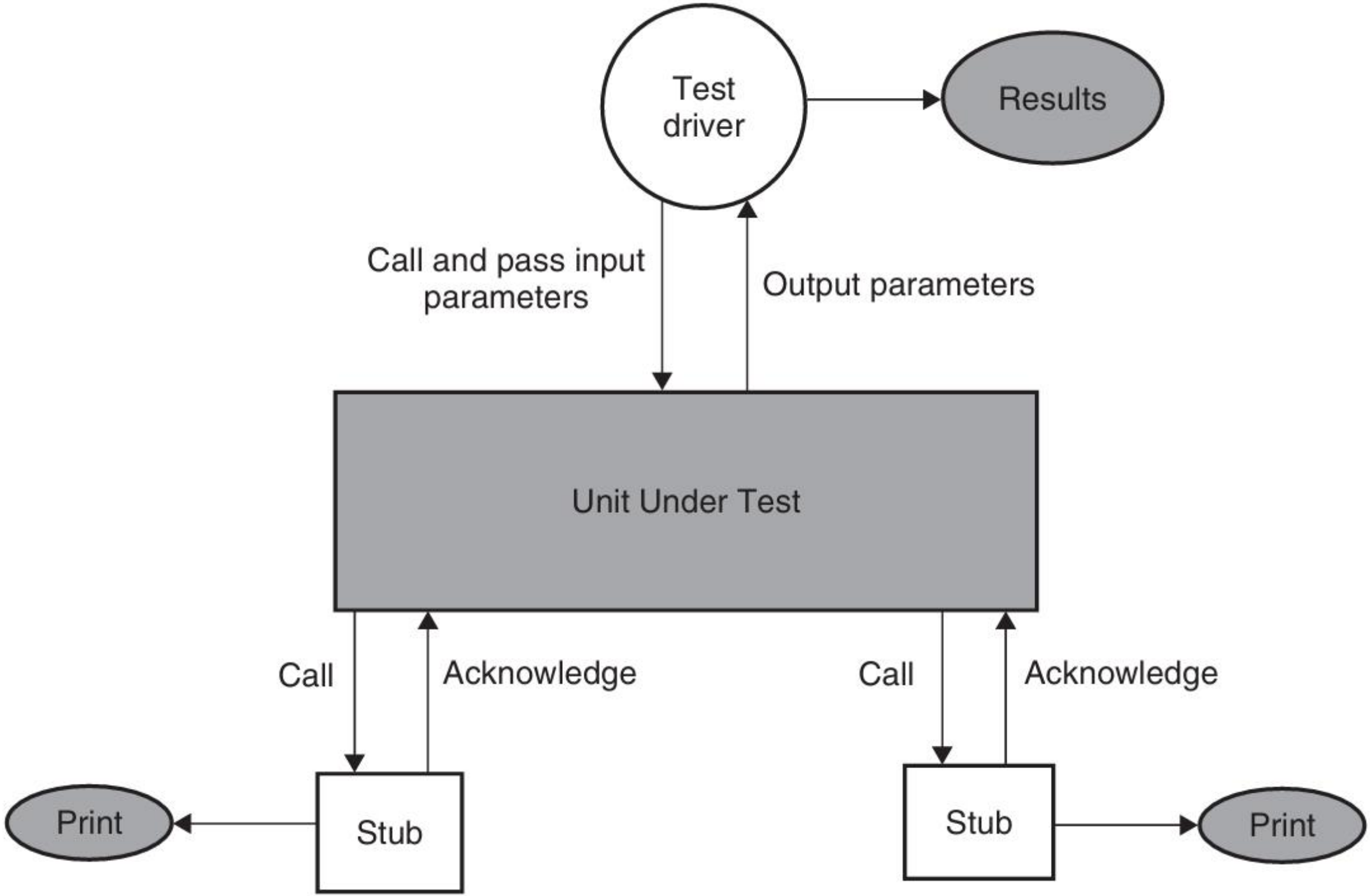
Context of the unit test consist of

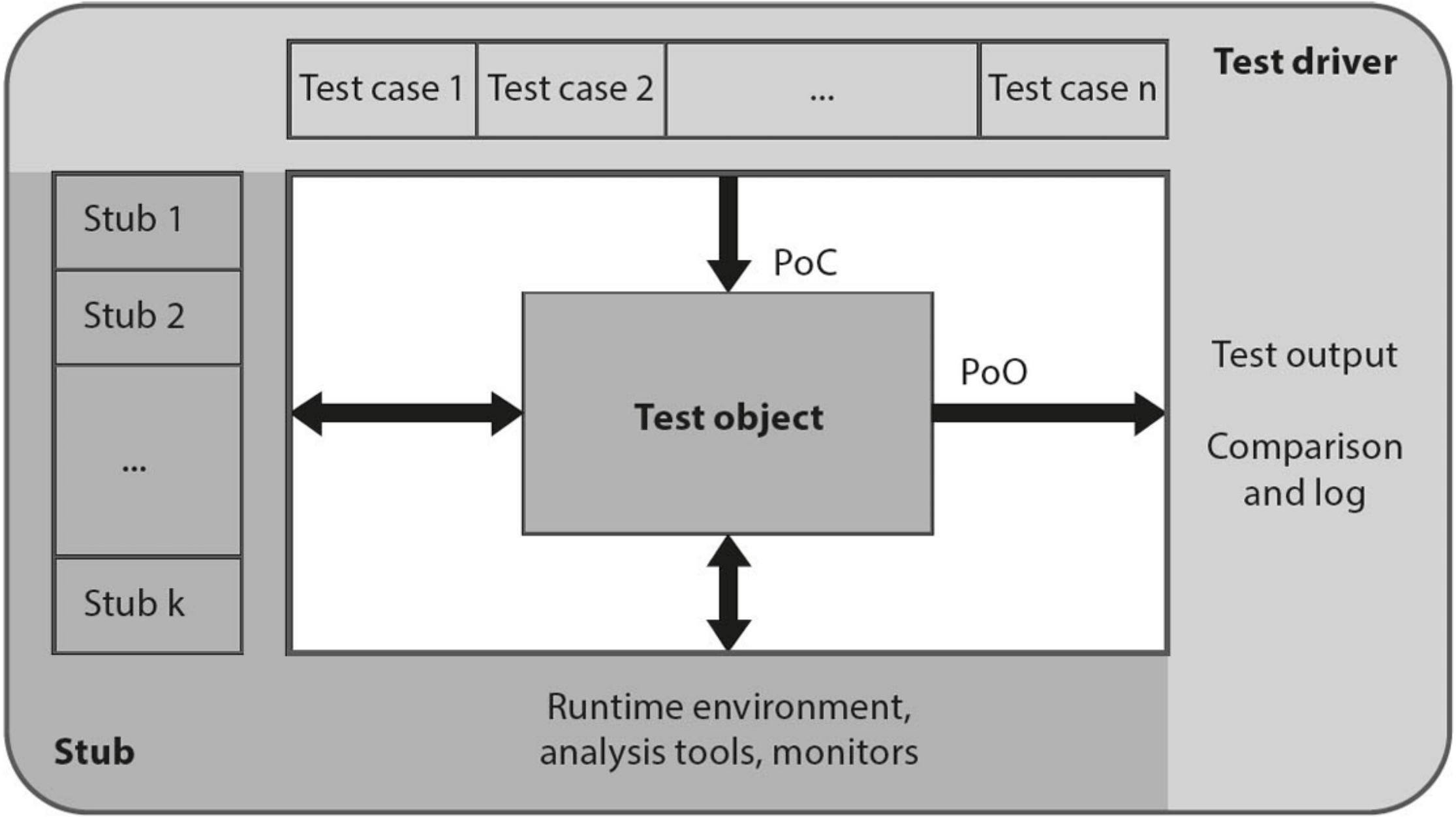
- Test Driver - Caller of the unit
- Stubs – Emulation of all the units called by the unit under test

Together form an executable test setup

Unit under test must be tested in isolation to accurately identify defects

The surrounding environment is emulated to avoid side effects and external influences


Test driver
Results
Call and pass input parameters
Output parameters
Unit Under Test
Call
Acknowledge
Call
Acknowledge
Print
Stub
Stub
Print


Test case 1
Test case 2
...
Test case n
Test driver
Stub 1
Stub 2
...
Stub k
PoC
Test object
PoO
Test output
Comparison and log
Runtime environment, analysis tools, monitors
Stub

POINT OF OBSERVATION AND CONTROL

Point of control (POC) – where you give input

This is the place in the system where the tester provides input or initiates an action

It is where you control the software behavior by simulating a user or external system

Example:

You enter a username and password into a login form

- The input field is a point of control because you are sending data to the system

POINT OF OBSERVATION AND CONTROL

Point of observation (POO) – where you check output

This is the place where you observe the system's output or reaction to the input

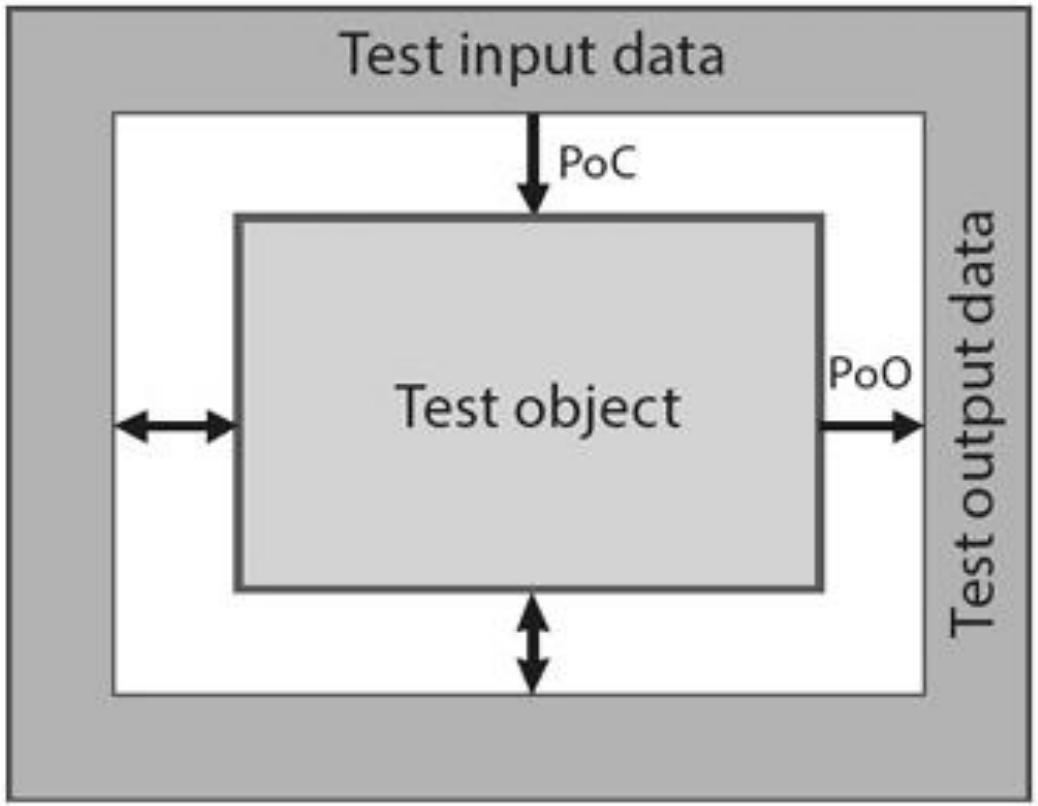
It's where you check if the result is correct

Example:

After submitting the login form, the system displays a welcome message or an error

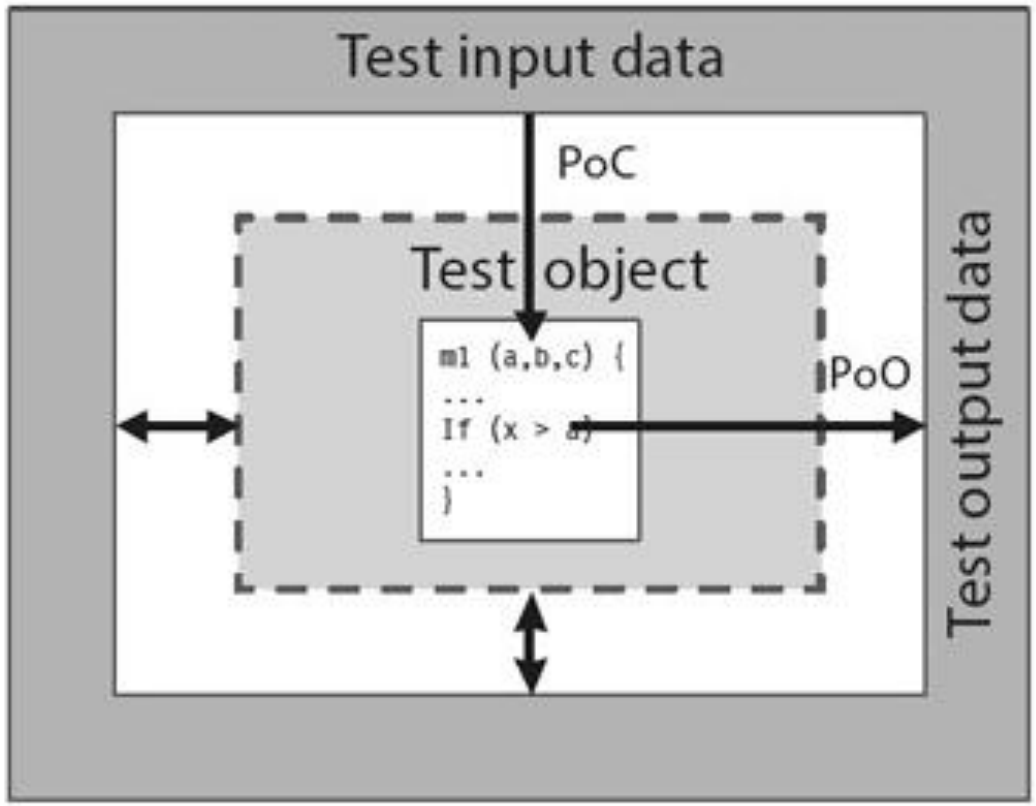
- This output screen is a point of observation because you observe the result here

Black-box technique



PoC and PoO
are "outside" the test object

White-box technique



PoC and/or PoO
are "inside" the test object

UNIT TEST FRAMEWORK CONT..

Test Driver

- Calls the unit under test
- Supplies input values and checks the actual result against expected output
- Acts as the main routine in this isolated environment
- Should be self-contained, with no reliance on external input files
- Must be capable of checking return values, internal variables, memory issues (leaks, allocation, deallocation), checks for file status (open/closed) and release other unused resources

Stubs

- Replaces any unit called by the unit under test (test object)
- Minimal implementation:
  - Print/log a message to confirm invocation
  - Return precomputed/dummy values to allow the test to proceed

UNIT TEST FRAMEWORK CONT..

Each test object must have its own dedicated test driver and stub

A shared test driver across multiple units is discouraged due to

- Increased complexity
- High coupling
- Risk of side effects from future changes

Drivers and stubs are not discarded after initial test

They are

- Maintained across the lifecycle of the unit
- Reused in regression testing
- Update upon code changes, new faults discovered and platform changes

BLACK BOX TESTING

Test cases are derived from specification due to the lack of knowledge about inner working of code

Also known as specification-based or behavior-based testing

All techniques that define test cases before coding (such as test-first/test-driven development) are by black box techniques

Point of observation for a black-box test is outside the test object and you need no knowledge of inner structure

Concentrate solely on the test object’s input and output behavior

DOMAIN TESTING

Large number of values in the input domain of the program and there are a large number of paths in a program

Partitioning the input into finite number of subdomains and assigning a distinct program path to each of the input subdomains

Program *P* where *D* is the domain of entire program which is divided into five subdomains D_1,... D_5. Part of program that specifies which code to execute for each subdomain is *input classifier*. Program does different computations for different subsets of its input domain

Domain can be represented by set of predicates

DOMAIN TESTING CONT..

A domain is defined, from a geometric perspective, by a set of constraints called boundary inequalities

P1:	$x + y > 5$	$\equiv$ *True*
P2:	$x <= -4$	$\equiv$ *True*

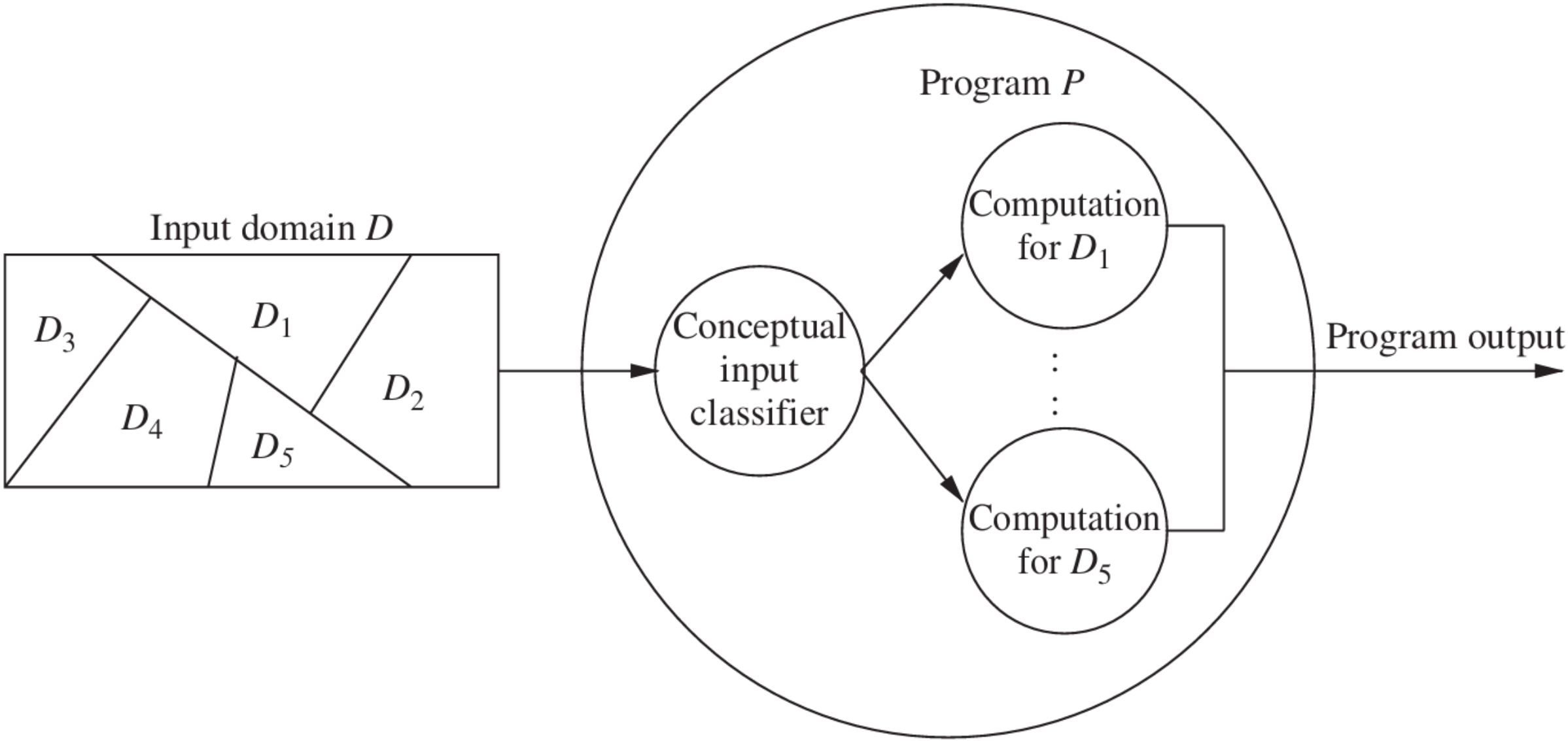
Closed boundary – If points on the boundary are included in the domain like P2 in above example

Open boundary – If points on the boundary do not belong to the domain like P1 in the above example

Equality symbol in a relational operator determines whether or not a boundary is closed

Closed domain – All boundaries are closed

Open domain – Some of the boundaries are open


Input domain D
D_3
D_1
D_2
D_4
D_5
Program P
Conceptual input classifier
Computation for D_1
Computation for D_5
Program output

PRACTICE

Given two predicates, identify and plot each domain:

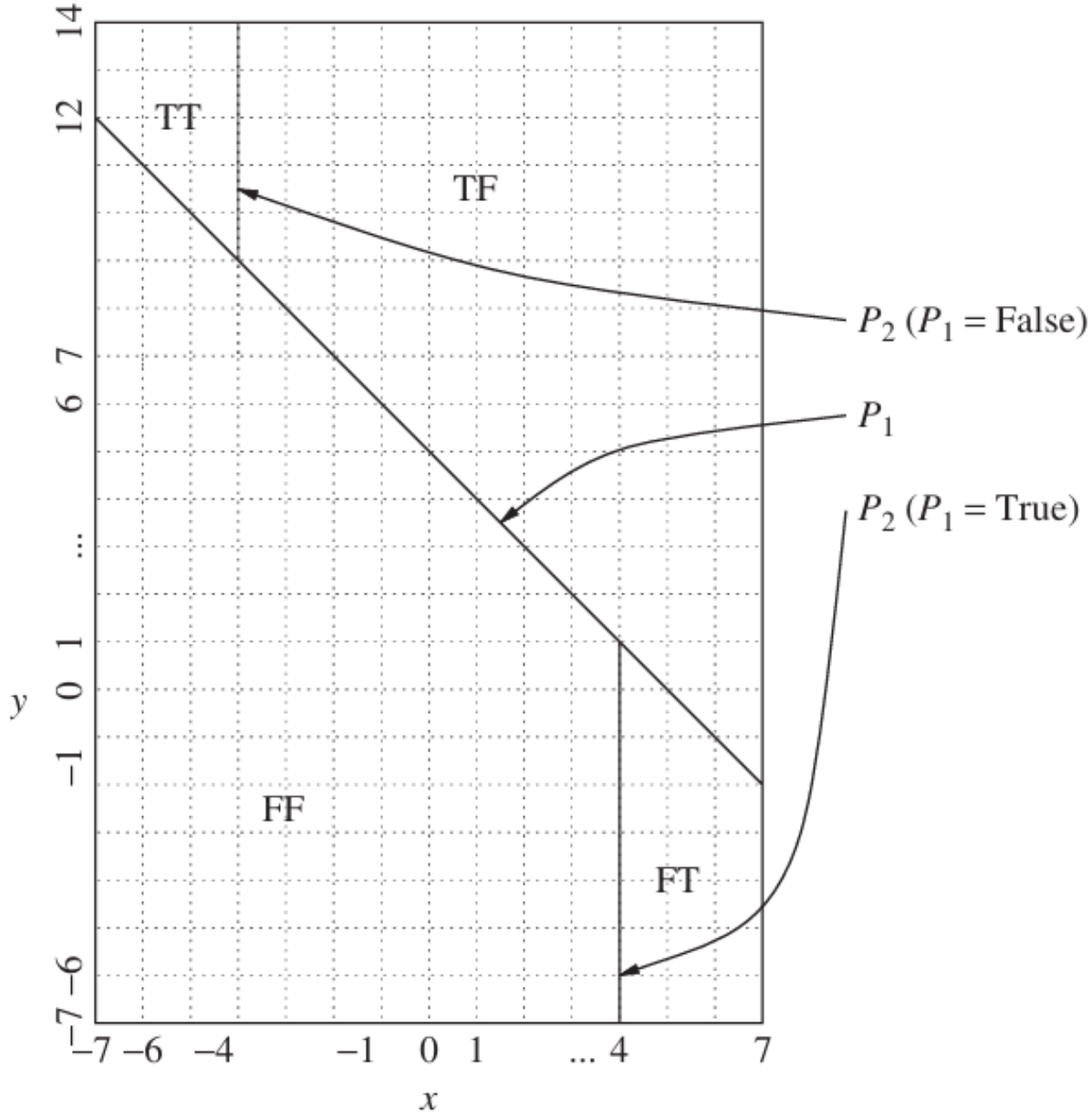
P1: $x + y > 5$

P2: $d >= c + 2$

```
int codedomain(int x, int y){
  int c, d, k
  c = x + y;
  if (c > 5) d = c - x/2;
  else       d = c + x/2;
  if (d >= c + 2) k = x + d/2;
  else            k = y + d/4;
  return(k);
}
```

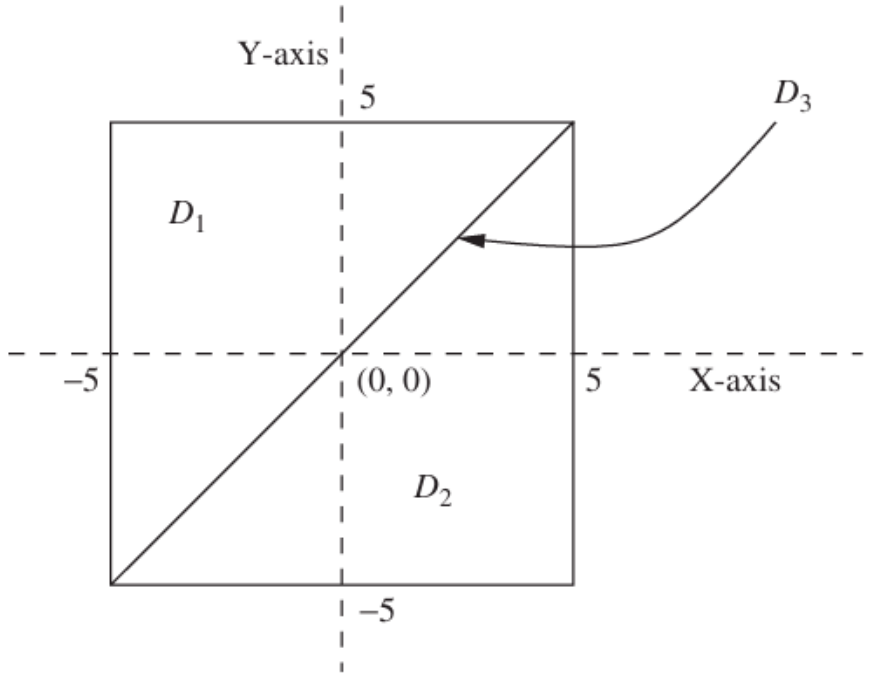
SOLUTION

Evaluation of P_1	Interpretation of P_2
True	$x \leq -4$
False	$x \geq 4$



PRACTICE

Consider the three domains D_1, D_2, and D_3 shown below. Domain D_3 consists of all those points lying on the indicated straight line. Assuming that the maximum X and Y span of all the three domains are [−5, 5] and [−5, 5], respectively, give concrete values of test points for domain D3.



EQUIVALENCE PARTITIONING

Equivalence partitioning is a black-box testing technique

Input data is split into partitions or classes

Each partition represents a set of values that should be treated the same by the system

EP should also applied on output instead of input only

Some requirements makes boundary of partition unclear (price <= 20 or price < 20)

Reduces number of test cases

Avoid closure error – placing (‘>=‘ instead of ‘>’)

Testing one representative value from each class is assumed sufficient

Type of partitions

- Valid Equivalence Partition (vEP)
- Invalid Equivalence Partition (iEP)

EQUIVALENCE PARTITIONING

The software enables the manufacturer to give its dealers various discounts. The corresponding requirement reads thus:

- For prices below $15,000 there is no discount. For prices up to $20,000, a discount of 5% is appropriate. If the price is below $25,000, a 7% discount is possible. If the price is above $25,000, a discount of 8.5% is to be applied. Four equivalence partitions containing valid input values (vEP) can easily be derived for calculating the discount.

Valid equivalence partition (vEP)

Parameter	Equivalence partitions	Representative
Price	vEP_1: $0 \leq x < 15000$ vEP_2: $15000 \leq x \leq 20000$ vEP_3: $20000 < x < 25000$ vEP_4: $x \geq 25000$	14500 16500 24750 31800

EQUIVALENCE PARTITIONING

Invalid equivalence partitions (iEP)

Parameter	Equivalence partitions	Representative
Price	iEP_1: $x < 0$ (negative price)	-4000
	iEP_2: $x > 1000000$ (unrealistically high price[3])	1500800

PRACTICE

Consider a software system that computes income tax based on adjusted gross income (AGI) according to the following rules

If AGI is between $1 and $29,500, the tax due is 22% of AGI.

If AGI is between $29,501 and $58,500, the tax due is 27% of AGI.

If AGI is between $58,501 and $100 billion, the tax due is 36% of AGI.

SOLUTION

There are three input conditions:

- Condition 1: $1 ≤ AGI ≤ $29,500
- Condition 2: $29,501 ≤ AGI ≤ $58,500
- Condition 3: $58,501 ≤ AGI ≤ $100 billion

First we consider condition 1 to derive two ECs:

- EC1: $1≤ AGI ≤ $29,500; valid input.
- EC2: AGI < 1; invalid input

Then, we consider condition 2 to derive on EC:

- EC3: $29,501 ≤ AGI ≤ $58,500; valid input

Finally, we consider condition 3 to derive two ECs:

- EC4: $58,501 ≤ AGI ≤ $100 billion; valid input.
- EC5: AGI > $100 billion; invalid input.

COVERAGE

An exit criteria can be defined by the relationship between number of tested values and the total number of equivalence partitions:

EP coverage = (Number of tested EPs / total number of EPs) x 100%

If 15 test cases are executed on 18 partitions, coverage will be:

EP coverage = (15/18) x 100% = 83.33%

BOUNDARY VALUE ANALYSIS

BVA complements equivalence partitioning by focusing on values at the *edges* (boundaries) of valid and invalid input ranges

Many bugs tend to cluster at boundaries due to off-by-one errors or incorrect comparison operators

BVA is only applicable when the input data can be ordered (numerically or logically)

For each boundary, three values are tested:

- The exact boundary value
- One value just inside
- One value just outside

$-10.0 \leq X \leq 10.0 = \{-9.9\ -10.0, -10.1\}$ and $\{9.9, 10.0, 10.1\}$

EXAMPLE

Let us consider the five ECs identified in our previous example to compute income tax based on AGI. The BVA technique results in test as follows for each EC. The redundant data points may be eliminated.

EC1: $1≤ AGI ≤ $29,500; This would result in values of $1, $0, $–1, $1.50 and $29,499.50, $29,500, $29,500.50.

EC2: AGI < 1; This would result in values of $1, $0, $–1, $–100 billion.

EC3: $29,501 ≤ AGI ≤ $58,500; This would result in values of $29,500, $29,500.50, $29,501, $58,499, $58,500, $58,500.50, $58,501.

EC4: $58,501 ≤ AGI ≤$100 billion; This would result in values of $58,500, $58,500.50, $58,501, $100 billion, $101 billion.

EC5: AGI > $100 billion; This would result in $100 billion, $101 billion, $10000 billion

COVERAGE

An exit criteria can be defined by the relationship between number of tested values and the total number of BVs:

BV coverage = (Number of tested BV / total number of BV) x 100%

If 15 test cases are executed on 18 BVs, coverage will be:

BV coverage = (15/18) x 100% = 83.33%

STATE TRANSITION TESTING

Black-box testing technique which focuses on

- The behavior of the system as it moves from one state to another
- The history of system inputs and actions, not just current input data
- Useful when system has finite number of states and output depends on prior events

A state machine has

- Finite number of states
- Events/inputs that trigger transitions between states
- Action tied to transitions
- Deterministic behavior (same input each time, same output)

Guard conditions are useful when multiple transitions are possible from a state, guard conditions define which one to take

STATE TRANSITION TESTING CONT..

Coverage criteria can be identified based on the available resources whether all states are required to test or only subset of critical state

N-switch coverage measures depth of test sequences

- 0-switch – one transition
- 1-switch – two transitions
- 2-switch – three transitions

Useful for testing loops/cycles in state machines

State transition testing can be used to test GUIs where each mask/dialog will be a state and user interaction (click, input) will be an event or transition

EXAMPLE

The vehicle support over-the-air (OTA) software updates, which can only occur when engine is off.

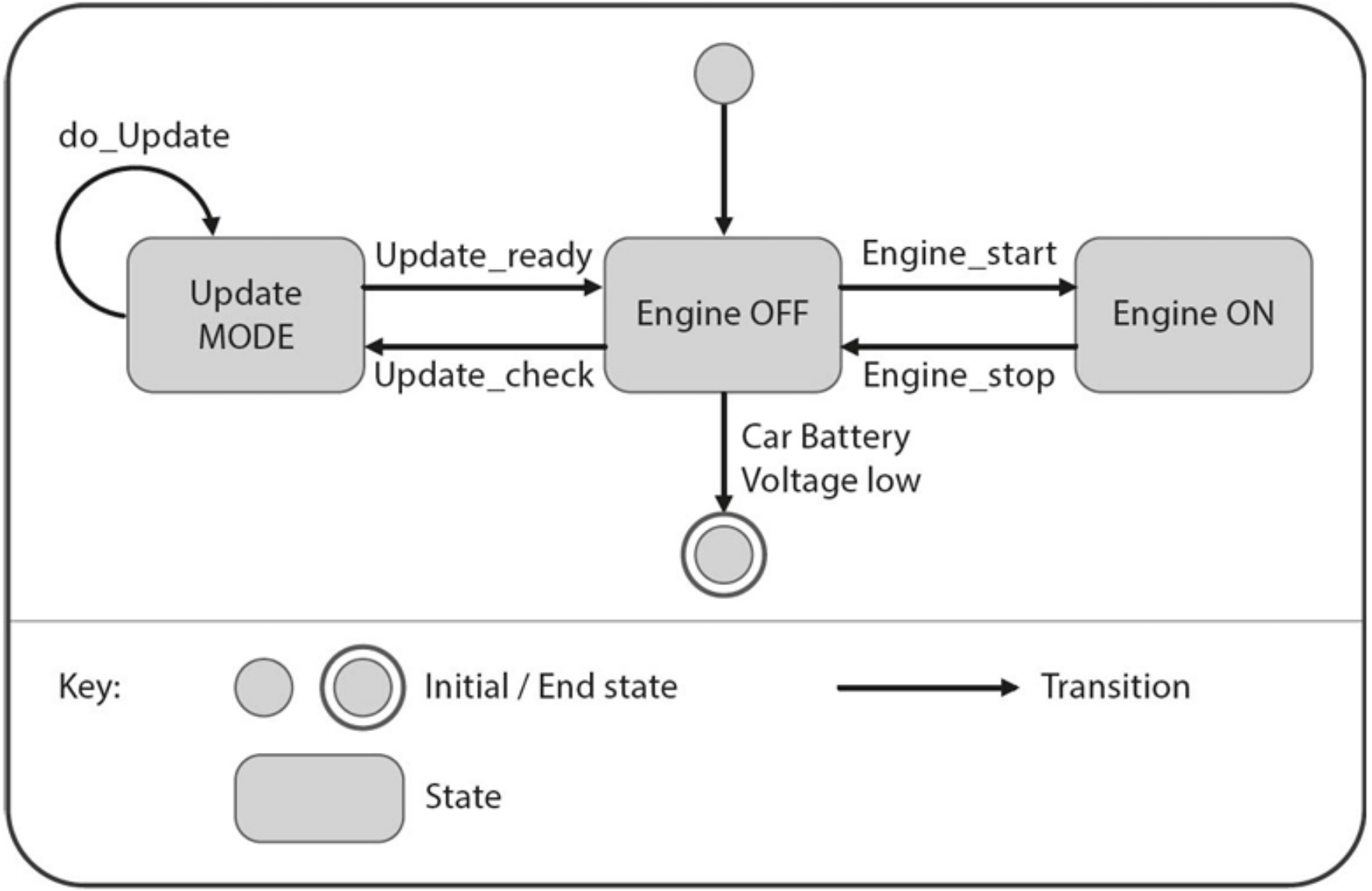
Engine OFF – Default idle state

Engine ON – Active driving state

Update MODE – State during OTA update

System enters *Engine OFF* state if battery is sufficient

System shuts down if the battery is depleted



State machine for a vehicle software update

Current state event	Engine OFF	Engine ON	Update MODE
Engine_start	Engine ON	–	–
Engine_stop	–	Engine OFF	–
Update_check	Update MODE	–	–
do_Update	–	–	Update MODE
Update_ready	–	–	Engine OFF

State transition table for a vehicle software update

DECISION TABLE TESTING

Focus on combinations of input conditions and their effects

EP and BVA only focuses on individual input instead of combination of input

Decision table can be drawn directly or with the help of cause and effect diagram

- Upper left – Conditions
- Upper right – All combinations of conditions
- Lower left – Actions
- Lower right – which actions occur for each combination

To create decision table

- Identify all conditions and actions
- List all combinations of conditions (2n for n conditions)
- Determine what actions follow each combination
- Remove redundant columns

EXAMPLE

Create decision table for the given scenario:

A special offer for a limited period is aimed at increasing vehicle sales. All standard models receive an 8% discount and all special editions (for which no optional extras are available) a 10% discount. If more than three extras are selected for a standard model, these receive an additional 15% discount. All other models receive no additional base discount and no extra discount for additional extras.

SOLUTION

Conditions

- Is it a standard model?
- Is it a special edition?
- Are more than 3 extras selected?

Actions

- 8% discount
- 10% discount
- 15% discount
- No discount

SOLUTION CONT..

Upper part of the decision table showing all combinations of conditions

Decision table		TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8
Conditions	Special edition?	Yes	Yes	Yes	Yes	No	No	No	No
	Standard model?	Yes	Yes	No	No	Yes	Yes	No	No
	Extras > 3	Yes	No	Yes	No	Yes	No	Yes	No

Decision table including “don’t care” cases

Decision table		TC1	TC2	TC3	TC4	TC5	TC6	TC7	TC8
Conditions	Special edition?	Yes	Yes	Yes	Yes	No	No	No	No
	Standard model?	–	–	–	–	Yes	Yes	No	No
	Extras > 3	–	–	–	–	Yes	No	–	–
Actions	10% discount	X	X	X	X				
	8% discount					X	X		
	15% discount for extras					X			
	No discount							X	X

SOLUTION CONT..

The table can now be consolidated. Test cases 1-4 are identical and can be treated as one, as can test cases 7 and 8. The result is the following decision table with 4 columns for 4 test cases

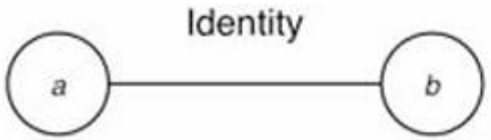
Decision table		TC1	TC5	TC6	TC7
Conditions	Special edition?	Yes	No	No	No
	Standard model?	–	Yes	Yes	No
	Extras > 3	–	Yes	No	–
Actions	10% discount	X			
	8% discount		X	X	
	15% discount for extras		X		
	No discount				X

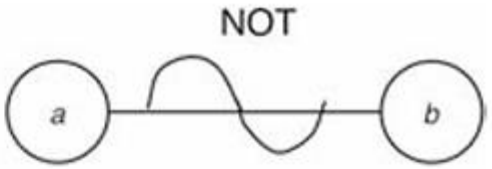
DECISION TABLE USING CAUSE-EFFECT GRAPHING

A cause-effect graph is a formal language into which a natural-language specification is translated

In order to create cause effect graph, follow the steps:

- Break down the specification
- Identify causes and effects
- Build the Boolean graph
- Add constraints (optional)
- Convert to decision table
- Generate test cases



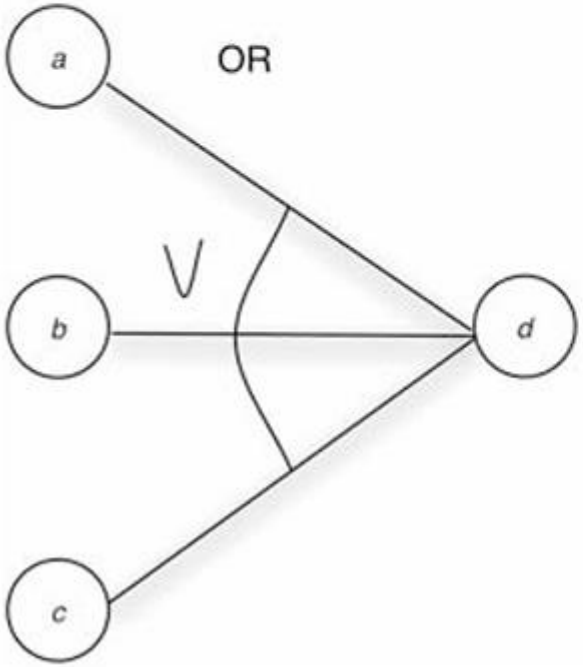


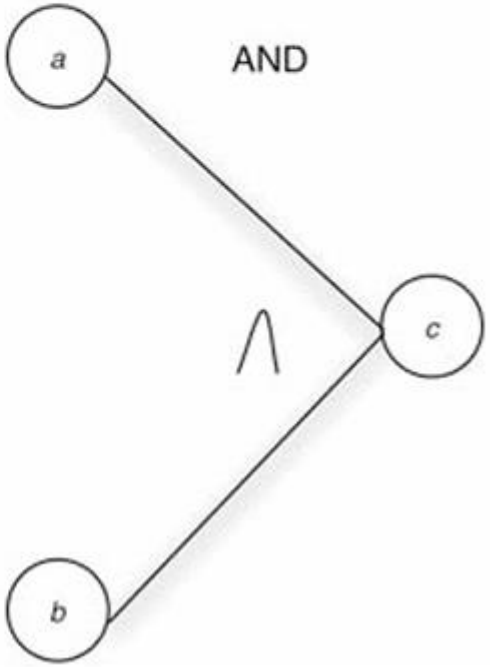
Identity (a → b): If a = 1, then b = 1; else b = 0

NOT: If a = 1, then b = 0; else b = 1

OR: If any input is 1, output is 1

AND: If all inputs are 1, output is 1





EXAMPLE

Generate cause-effect graph for the given scenario:

The character in column 1 must be an “A” or a “B.” The character in column 2 must be a digit. In this situation, the file update is made. If the first character is incorrect, message X12 is issued. If the second character is not a digit, message X13 is issued.

SOLUTION

The *causes* are

1—character in column 1 is “A”

2—character in column 1 is “B”

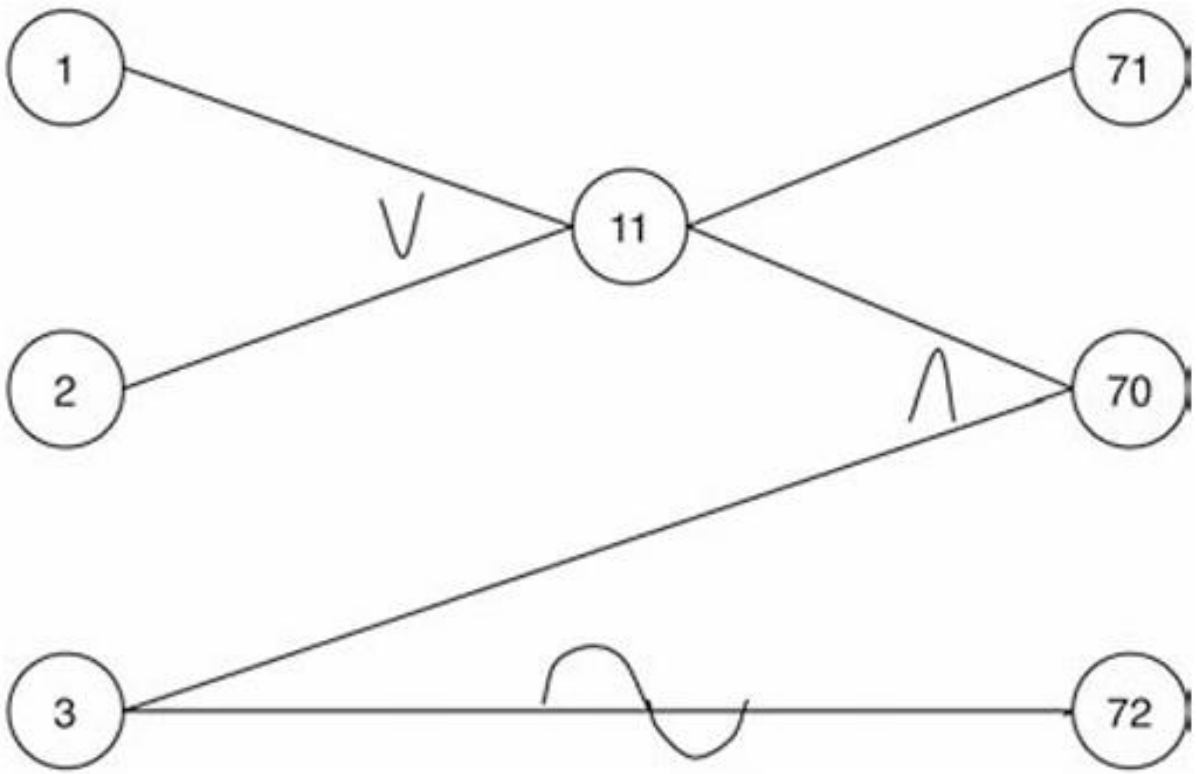
3—character in column 2 is a digit

and the *effects* are

70—update made

71—message X12 is issued

72—message X13 is issued



EXAMPLE

Consider the withdrawing cash from an ATM scenario and draw cause-effect diagram and decision table:

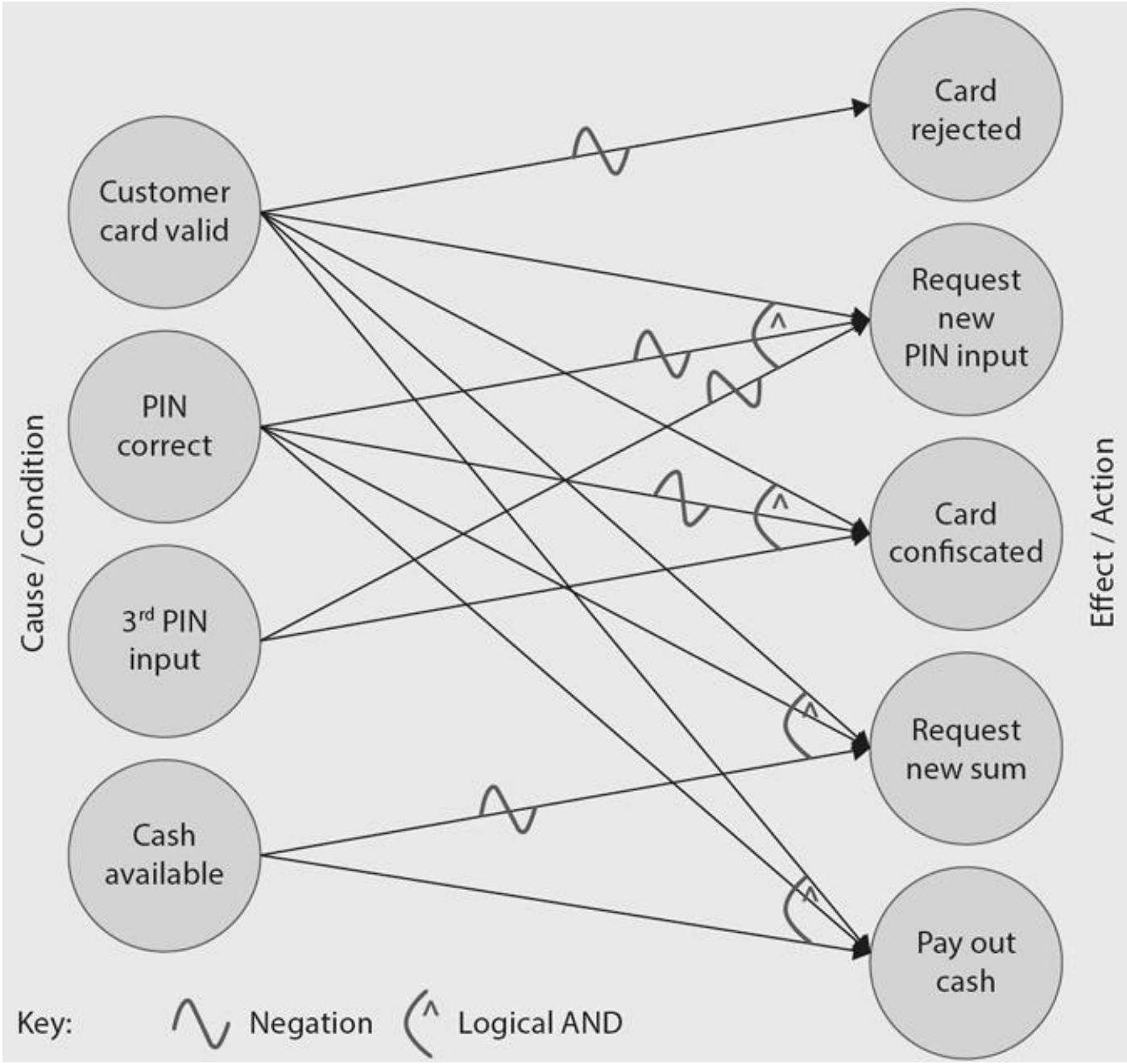
Conditions:

- The customer’s card is valid
- The correct PIN is entered
- A maximum of three PIN input attempts is allowed
- Money is available (in the account and in the ATM)

The possible (re)actions of the ATM are as follows:

- The card is rejected
- The user is asked to re-enter the PIN
- The card is confiscated
- The user is asked to enter a different sum of money
- Money is paid out

SOLUTION


Customer card valid
PIN correct
3rd PIN input
Cash available
Cause / Condition
Card rejected
Request new PIN input
Card confiscated
Request new sum
Pay out cash
Effect / Action
Key:
Negation
Logical AND

SOLUTION

Decision table		TC1	TC2	TC3	TC4	TC5
Conditions	Customer Card valid?	No	Yes	Yes	Yes	Yes
	PIN correct?	–	No	No	Yes	Yes
	3rd PIN input?	–	No	Yes	–	–
	Cash available?	–	–	–	No	Yes
Actions	Card rejected	X				
	Request new PIN input		X			
	Card confiscated			X		
	Request new sum				X	
	Pay out cash					X

PAIRWISE TESTING

Each possible combination of values for every pair of input variables is covered by at least one test case

Test every possible pair of input parameter values at least once

Most bugs are caused by the interaction of two parameters rather than all of them together

For 3x3 Boolean input (A, B, C), there are 8 total combinations

With pairwise testing, only 4 combinations might suffice if each pair (A-B, B-C, A-C) covers all 2x2 value combinations

Orthogonal array – Every pair occurs same number of times (large)

Covering array – every pair occurs at least once (small)

N-WISE TESTING

Generalization of pair-wise: test combinations of n parameters at a time

For example, testing all 3-way combinations (n=3) would cover failures that occur due to interactions between three parameters

Objective of pairwise testing is to identify failures that occur due to the interaction of pairs of parameters

Inputs: 10 models, 5 engines, 10 rims × 2 tire types, 10 colors × 3 paint effects, 5 entertainment systems

Total combinations: 150,000

Testing all the possible variants would take **1.7 days** of testing time

N-WISE TESTING CONT..

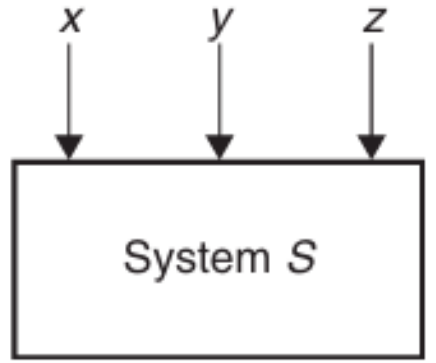
Higher value of n increases number of test cases that have to be designed and executed

In order to achieve 100% coverage, all test cases have to be executed

If the value of n is very high, achieving 100% coverage involves a lot of effort and often is not a practical solution

Solution: Increase n by 1 once a failure has been discovered and remedied, and to repeat the process until no more failures occur on the next level up

EXAMPLE



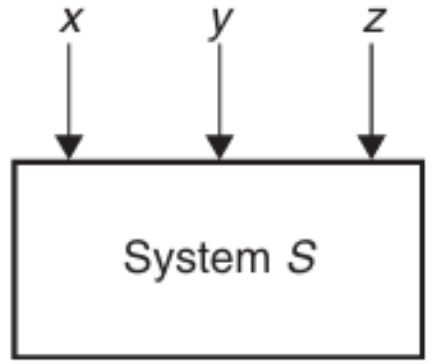
Consider the system *S*, which has three input variables X,Y, and Z. Let the notation D(w) denote the set of values for an arbitrary variable 'w'. For the three given variables X, Y, and Z, their value sets are as follows:

D(X)={True, False}, D(Y) ={0,5}, and D(Z) ={Q,R}

Test Case ID	Input *X*	Input *Y*	Input *Z*
TC_1	True	0	*Q*
TC_2	True	5	*R*
TC_3	False	0	*Q*
TC_4	False	5	*R*

Pairwise test cases for system S

EXAMPLE CONT..



Choose any two columns at random and find all pairs (not all combinations of 1's and 2's). This is an example of $L_4(2^3)$ orthogonal array. The 4 indicates 4 rows (runs), 2^3 indicates 3 columns (factors), number of possible values in a column (level)

Orthogonal arrays are denoted by L_{Runs} $(Levels^{Factors})$

	Factors		
Runs	1	2	3
1	1	1	1
2	1	2	2
3	2	1	2
4	2	2	1

$L_4(2^3)$ orthogonal array

EXAMPLE

Consider a website that is viewed on a number of browsers with various plug-ins and operating systems (OSs) and through different connections as shown in Table below. The table shows the variables and their values that are used as elements of the orthogonal array. We need to test the system with different combinations of the input values

Variables	Values
Browser	Netscape, Internet Explorer (IE), Mozilla
Plug-in	Realplayer, Mediaplayer
OS	Windows, Linux, Macintosh
Connection	LAN, PPP, ISDN

Various values that need to be tested in combination

SOLUTION

Map the variables to the factors and values to the levels of the array: The factor 1 to Browser, the factor 2 to Plug-in, the factor 3 to OS, and the factor 4 to Connection. Let 1 = Netscape, 2 = IE, and 3 = Mozilla in the Browser column. In the Plug-in column, let 1 = Real player and 3 = Media player. Let 1=Windows, 2=Linux, and3=Macintosh in the OS column. Let 1 = LAN, 2 = PPP, and 3 = ISDN in the Connection column

$L_9(3^4)$ Orthogonal Array

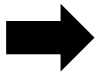
	Factors			
Runs	1	2	3	4
1	1	1	1	1
2	1	2	2	2
3	1	3	3	3
4	2	1	2	3
5	2	2	3	1
6	2	3	1	2
7	3	1	3	2
8	3	2	1	3
9	3	3	2	1

SOLUTION CONT..

Start at the top of the Plug-in column and cycle through the possible values when filling in the left-over levels

(3^4) Orthogonal Array after Mapping Factors

Test Case ID	Browser	Plug-in	OS	Connection
TC_1	Netscape	Realplayer	Windows	LAN
TC_2	Netscape	2	Linux	PPP
TC_3	Netscape	Mediaplayer	Macintosh	ISDN
TC_4	IE	Realplayer	Linux	ISDN
TC_5	IE	2	Macintosh	LAN
TC_6	IE	Mediaplayer	Windows	PPP
TC_7	Mozilla	Realplayer	Macintosh	PPP
TC_8	Mozilla	2	Windows	ISDN
TC_9	Mozilla	Mediaplayer	Linux	LAN



Generated Test Cases after Mapping Left-Over Levels

Test Case ID	Browser	Plug-in	OS	Connection
TC_1	Netscape	Realplayer	Windows	LAN
TC_2	Netscape	Realplayer	Linux	PPP
TC_3	Netscape	Mediaplayer	Macintosh	ISDN
TC_4	IE	Realplayer	Linux	ISDN
TC_5	IE	Mediaplayer	Macintosh	LAN
TC_6	IE	Mediaplayer	Windows	PPP
TC_7	Mozilla	Realplayer	Macintosh	PPP
TC_8	Mozilla	Realplayer	Windows	ISDN
TC_9	Mozilla	Mediaplayer	Linux	LAN

USE CASE TESTING

Use cases describe interactions between a user (or external system) and the system under test (SUT)

Used for identifying and documenting system requirements

Commonly illustrated using use case diagrams, which depict typical user/system interaction

Useful for system testing, acceptance testing and occasionally for integration testing

Use case includes

- Preconditions – must be true before execution (e.g. customer must logged in)
- Post conditions – outcomes after use case runs (e.g. order placed)

Use cases represents normal behavior and alternatives

USE CASE TESTING CONT..

Ideal for checking typical user scenarios

Helps verify that the system behaves correctly under normal conditions

Test cases should cover

- All main flows
- All alternatives (e.g. ‘extend’ paths)

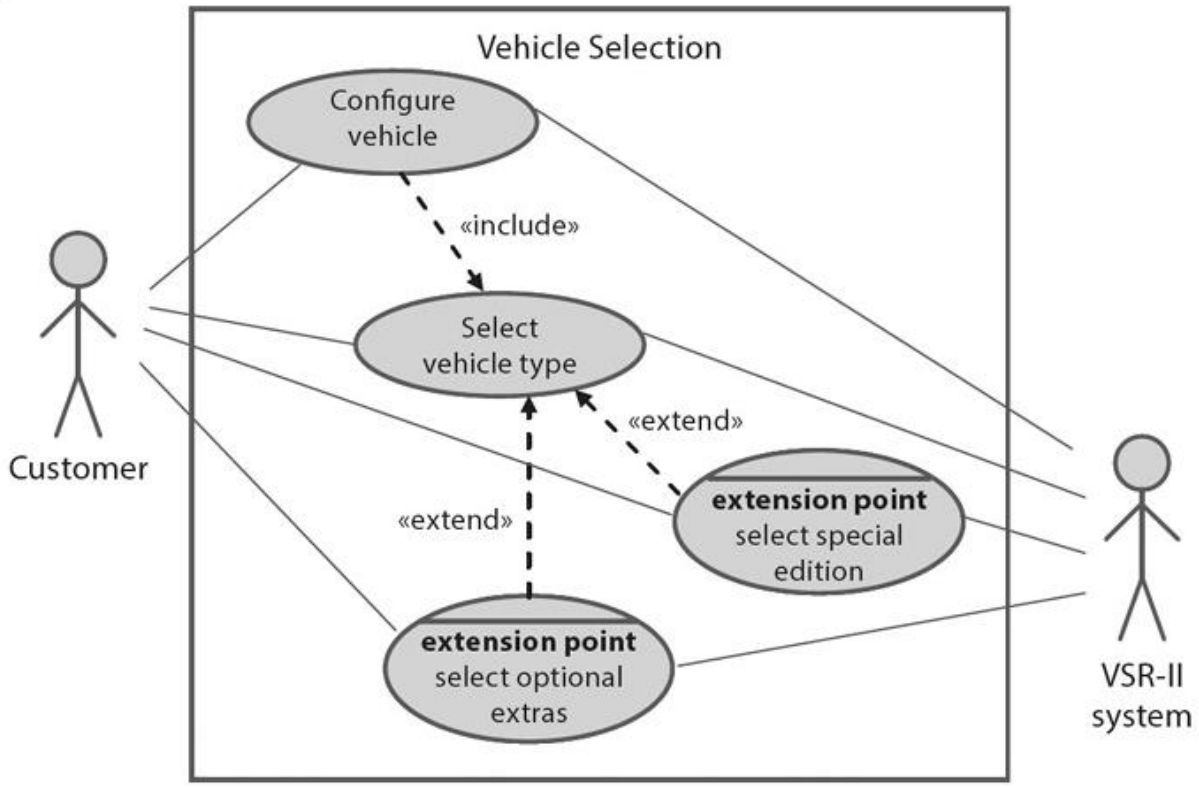
Test design requires

- Initial state and preconditions
- Constraints
- Expected outcome and post conditions

Exit criteria is that every use case must be covered by at least one or more test cases

USE CASE TESTING CONT..

To configure a vehicle, a customer has to select a vehicle type. Once this has happened, there are three alternative ways to proceed. The customer can select a special edition, optional extras, or neither of these.



OTHER BLACK BOX TESTING TECHNIQUES

- Syntax Testing
- Random Testing
- Smoke Testing

SYNTAX TESTING

Ensures the system accepts valid input formats and rejects incorrect ones

Based on formally defined syntax rules (grammar rules, data formats)

Example: Testing whether a form rejects improperly formatted email addresses

RANDOM TESTING

Checks system reliability using randomly chosen inputs

Random values are selected (ideally with a statistical distribution like normal distribution)

Helps find unexpected system behavior under varied input conditions

SMOKE TESTING

Quickly checks if the system’s core functions work and it doesn’t crash

Runs a basic set of tests, often automated and reused

Performed early, especially after builds/updates, to decide if deeper testing is worth proceeding with

Named after the idea that if it “doesn’t go up in smoke”, it might be usable!

MORE ON BLACK BOX TESTING

+ Focuses on user-facing functionality
+ Ensures the system behaves as expected in common usage scenarios
+ Essential for system and acceptance testing

MORE ON BLACK BOX TESTING

- If the requirements themselves are wrong, black-box testing won’t catch it
- The system may *pass* tests based on incorrect assumptions
- Only verifies what’s explicitly stated, not what might be unintentionally present
- Primarily checks functional correctness, no internal structure or code quality
- Doesn’t assess how well code is written – just it works

WHITE BOX TESTING

Focus on the structure and behavior within the test object

Also known as structured-based testing or code-related testing

The point of observation (PoC) lies within the test object

Test cases can be derived from the structure of the application’s code or specification

Objective is to

- Verify a specified degree of *structural coverage* i.e. 80% of the statements in the test object need to be covered by test cases
- Ensure every individual statement in code runs at least once

WHITE BOX TESTING CONT..

We can differentiate between different types of white-box test techniques:

- Statement Testing
- Decision Testing
- Condition Testing
  - Branch condition testing
  - Branch condition combination testing
  - Modified condition decision coverage testing
- Path Testing

STATEMENT TESTING

White box testing technique that focuses on executing each individual statement in the program at least once

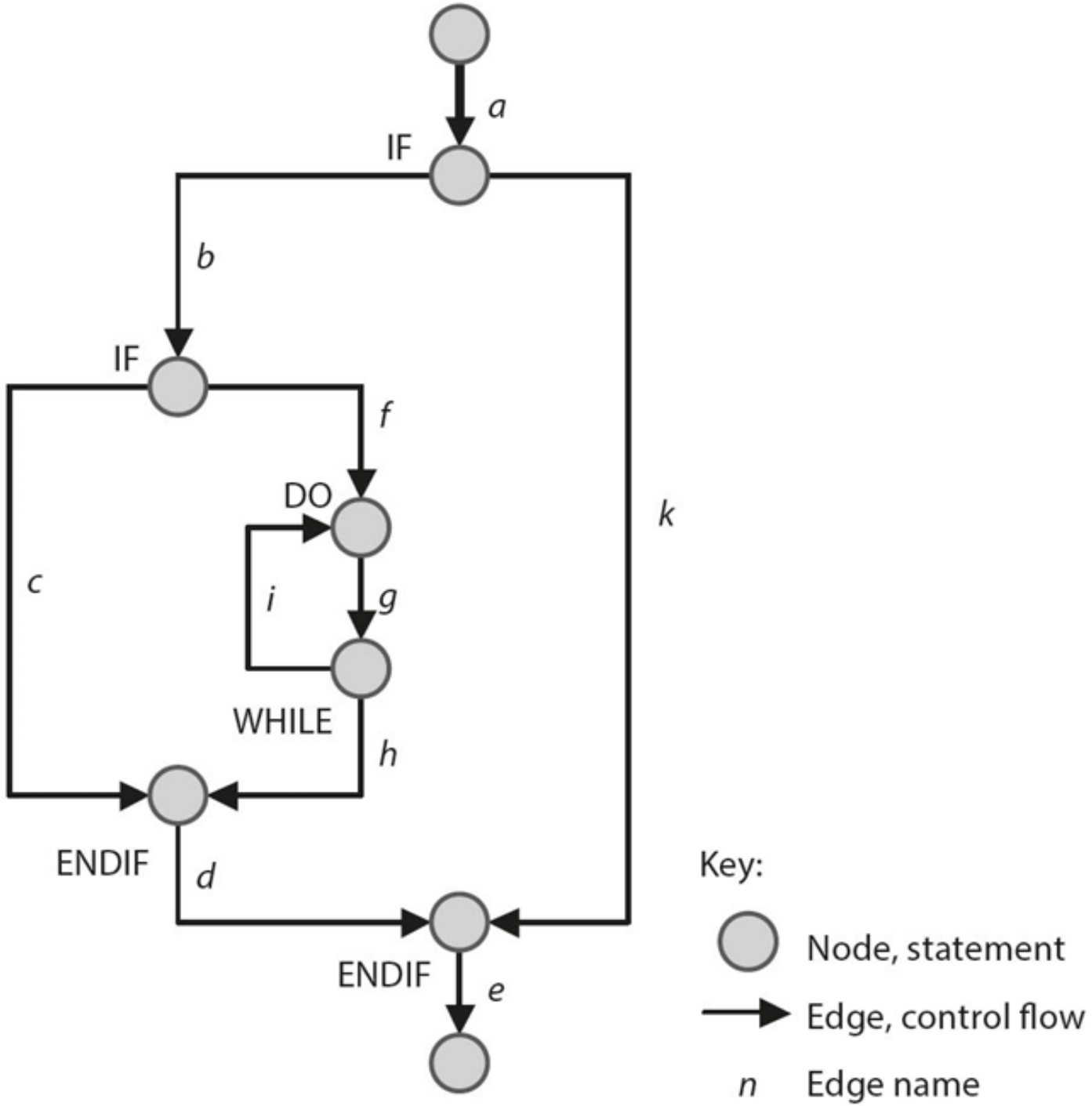
The goal is to ensure that all code lines are tested and checked for correctness

Control flow graph is created from code

- Nodes – statements
- Edges – control flow (path between statements)
- Unconditional statements are grouped into a single node

This example is based on a simple piece of code that contains just two IF-statements and a single loop

The test case has to traverse all the edges in the following order:
a, b, f, g, h, d, e



STATEMENT TESTING CONT..

The *exit criteria* for this type of test can be clearly defined using the following formula:

Statement coverage = (number of executed statements / total number of statements)×100%

This is called *C0 Coverage*

It’s considered a basic/weak form of code coverage

Testers aim for 100%, but in practice, some unreachable code may make this impossible without excessive effort

DECISION TESTING

Ensure that each possible decision outcome (True/False) in code is tested

Each decision (IF, CASE, loop) creates branches (edges in control flow)

Tests must cover all edges (all possible decision outcomes)

Decision Coverage = (Tested Decision Outcomes / Total Decision Outcomes) x 100%

This is called *C1 coverage*

Decision testing detects missing logic in IF/ELSE branches

Better for identifying logic errors than C0

Requires more test cases than statement testing

CONDITION TESTING

- Branch Condition Testing
- Branch Condition Combination Testing
- Modified Condition Decision Coverage

BRANCH CONDITION TESTING

Evaluate each atomic condition (e.g. x > 3, not x > 3 OR y < 5) for both true and false

Does not guarantee different outcomes of the full decision and hence weaker than decision testing

Example:

For **x > 3 OR y < 5**:

Test x = 6, y = 8 → T OR F → T

Test x = 2, y = 3 → F OR T → T

Both result in T, so we don't know if a bug would affect the overall logic.

BRANCH CONDITION COMBINATION TESTING

Test all combinations of conditions in a decision

Subsumes statement and decision testing. Ensures all logical combinations are exercised

Downside – exponential growth 2n combinations for n conditions

Not all combinations may be realizable due to logical constraints

BRANCH CONDITION COMBINATION TESTING

Condition: **x > 3 OR y < 5**

Example (unrealizable):

For $3 \leq x$ AND $x < 5$, you can’t test (F, F) since a number can't be both < 3 and ≥ 5.

Example (realizable)

x = 6 (T), y = 3 (T), x > 3 OR y < 5 (T)

x = 6 (T), y = 8 (F), x > 3 OR y < 5 (T)

x = 2 (F), y = 3 (T), x > 3 OR y < 5 (T)

x = 2 (F), y = 8 (F), x > 3 OR y < 5 (F)

MODIFIED CONDITION DECISION COVERAGE

For each condition, ensure that changing its value alone causes a change in the overall decision

Tests only combinations that matter (affect the outcome)

More efficient than full combination testing

Preferred for compound decisions

Example:

For **x > 3 OR y < 5**:

x = 6, y = 8 → T OR F → T

x = 2, y = 3 → F OR T → T

x = 2, y = 8 → F OR F → F

PATH TESTING

Ensures that all possible execution paths through a program are tested

This is more rigorous than statement or branch testing because it covers all combinations of branches and loops

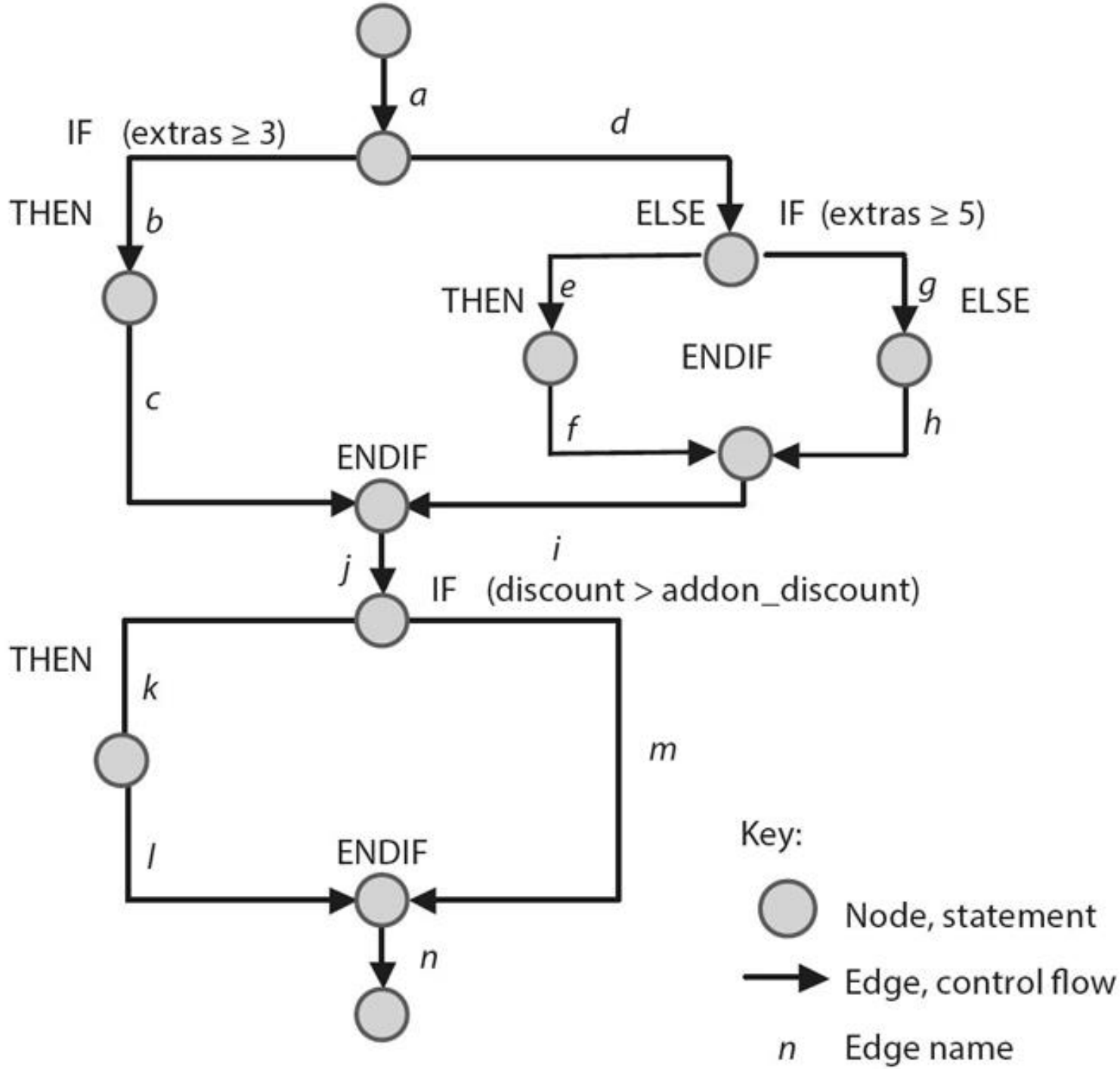
In simple statement or branch testing, loops might be executed once or not at all

Path testing forces you to test

- Loop not taken
- Loop taken once
- Loop taken multiple times

This is crucial because bugs often appears under repeated execution conditions

```
double calculate_price(double baseprice, double specialprice, double extraprice, int extras,  double discount) {
    double addon_discount, result;
    if (extras ≥ 3)
        addon_discount = 10;
    else
        if (extras ≥ 5)
            addon_discount = 15;
        else
            addon_discount = 0;
    if (discount > addon_discount)
        addon_discount = discount;
    result = baseprice/100.0 * (100-discount) + specialprice + extraprice/100.0 * (100-addon_discount);
    return result;
}
```


a
IF (extras ≥ 3)
d
THEN
b
ELSE
IF (extras ≥ 5)
THEN
e
g
ELSE
ENDIF
c
f
h
ENDIF
j
i
IF (discount > addon_discount)
THEN
k
m
l
ENDIF
n
Key:
Node, statement
Edge, control flow
n
Edge name

PATH TESTING CONT..

Test Case 1

price = calculate_price(10000.00,2000.00,1000.00,3,0);

test_ok = test_ok && (abs (price-12900.00) < 0.01);

Test Case 2

price = calculate_price(25500.00,3450.00,6000.00,6,0);

test_ok = test_ok && (abs (price-34050.00) < 0.01);

These test cases cause the following paths through the graph to be executed:

Test case 01: a, b, c, j, m, n

Test case 02: a, b, c, j, m, n

These test cases only execute two out of six possible decision outcomes, thus achieving 33% decision coverage (and 6/14, or **43%** branch coverage)

PATH TESTING CONT..

To improve coverage, following test cases are specified

Test Case 03

price = calculate_price(10000.00,2000.00,1000.00,0,10);

test_ok = test_ok && (abs (price-12000.00) < 0.01);

Test Case 04

price = calculate_price(25500.00,3450.00,6000.00,6,15);

test_ok = test_ok && (abs (price-30225.00) < 0.01);

These test cases cause the following paths through the graph to be executed:

Test case 03: a, d, g, h, i, j, k, l,

Test case 04: a, b, c, j, k, l, n

These test cases also execute the edges d, g, h, i, k, and l, and thus increase coverage to 5/6 decision outcomes = **83%** (and branch coverage of 12/14 = 86%). In this case, edges e and f are not executed.

EXPERIENCE BASED TESTING

Leverage the know-how and experience of testers, developers, users and other stakeholders to design test cases, test conditions and test data

Degree of coverage is difficult to ascertain and therefore cannot be effectively used as an exit criteria

Often used in conjunction with black and white box testing

INTUITIVE TEST CASE DERIVATION

A non-methodical approach to test case design

Relies on tester’s intuition, experience and knowledge

Often referred to as *error guessing*

Test cases are designed based on

- Past experience with defects
- Knowledge of common developer mistakes
- Insight into system weaknesses

No formal rules or steps

Tester guesses where error might be based on

- Past issues in similar systems
- Frequently misused functionalities

Helpful when documentation is limited

CHECK LIST BASED TESTING

Track past bugs, failures, and edge cases

Update regularly with new experiences

Helps in

- Avoiding repeated bugs
- Creating better test cases
- Remind the tester to cover all aspects

Coverage – how many checklists are tested

Use checklists to derive test cases

Build and maintain checklists overtime

EXPLORATORY TESTING

Used when no proper documentation and more time available

No predefined plan, tests evolve as knowledge grows

Elements of test object and their tasks and functions are ‘explored’ before deciding which elements to test

A virtual model of the system is built in the tester’s mind

Helps in identifying unexpected behavior

Drives new tests based on earlier results

Not a primary (formal) testing technique

VSR-II testers used the knowledge from testing older versions (VSR-I)

SESSION BASED EXPLORATORY TESTING

Uses time-boxing (e.g. 90 minutes per session)

Each session has

- A goal (called a ‘Test Charter’)
- Defined scope and focus
- All finding recorded in session sheets

Each *test charter* should answer

- Why are we testing?
- What are we testing?
- How are we testing?
- What problems are we looking for?

CHOOSING RIGHT TESTING TECHNIQUE

Many testing techniques exist – which one to use and when?

Some techniques are better suited for specific test levels

Its better to combine techniques for

Better coverage

Efficient use of time and budget

Techniques vary from informal to formal

CHOOSING RIGHT TESTING TECHNIQUE

Choosing the right technique depends on

- Project needs
- Team skills
- Regulations
- Time and budget
- Risk level
- Development model

CHOOSING RIGHT TESTING TECHNIQUE

1. Type of test object

Component level
- Use black-box and white box techniques

System level
- Use use-case or state-transition testing

User interface
- Focus on non-functional tests like usability

2. Complexity of the system

Simple logic: Basic decision testing is enough

Complex logic: Requires compound condition testing

More complexity – more thorough technique required

CHOOSING RIGHT TESTING TECHNIQUE

3. Adherence to standards

Some industries mandate specific techniques

Especially in safety-critical or regulated environments

Often includes coverage requirements (e.g., 100%)

4. Customer or contractual requirements

Customers may demand:

- Specific techniques
- Minimum test coverage

Ensures smoother acceptance testing

CHOOSING RIGHT TESTING TECHNIQUE

5. Testing objectives

Objectives may include:

- Correctness
- Completeness
- Architecture validation

Choose techniques that align with specific goals

6. Risk Analysis

High-risk components need:

- More testing
- Diverse techniques

Lower-risk – possibly simpler testing.

CHOOSING RIGHT TESTING TECHNIQUE

7. Planned software usage

Example:

- Simple web form = light testing
- Hospital power control system = intensive testing

Usage risk guides the depth and variety of testing

8. Available documentation

Good specs: Can automate test design

No/poor docs: Use exploratory testing to learn & test

CHOOSING RIGHT TESTING TECHNIQUE

9. Available tools

Tools help with:

- Test case design
- Execution
- Defect tracking

Choose tool-supported techniques for efficiency

10. Knowledge and skills

Choose techniques the team is skilled in

If not, provide training

Creative testers may prefer exploratory testing

CHOOSING RIGHT TESTING TECHNIQUE

11. Types of expected faults

Anticipate error types

Example: Range errors → Use boundary value analysis

Matching technique to likely fault – better detection

12. Development model

Agile: Needs frequent, automated tests.

V-model: Predefined levels → apply specific techniques at each level.

CHOOSING RIGHT TESTING TECHNIQUE

13. Time and budget constraints

Limited resources – Pick efficient techniques

Automation can save time and money

CONCLUSION

Correct functionality is of central importance to any software system, so thorough testing of the test object's functionality needs to be guaranteed

No *one-size-fits-all* technique exist in testing different aspects

Using equivalence partitioning and boundary value to derive test cases is recommended for every test object

If dependencies between input values need to be tested, use decision tables to document the dependencies and derive appropriate test cases

On the system testing level, use cases can be used as a basis for deriving test cases

White-box techniques are best applied to low-level tests, while black-box techniques can be applied on all levels